Análise de algoritmos

Aula 02 - Estrutura de dados

Professor Daniel Marques

UEPB
Universidade
Estadual da Paraíba

Busca sequencial e consumo de tempo

Quantos segundos demora para executar a seguinte função?

```
int busca(int [] v, int n, int x) {
    int i;
    for (i = 0; i < n; i++){
        if (v[i] == x) {
            return i;
        }
    }
    return -1;
}
```

Depende ...

Busca sequencial e consumo de tempo

Depende do computador onde ele for rodado: rápido vs lento

Depende de x no vetor

- No melhor caso, a linha 4 é executada 1 vez
- No pior caso, a linha 4 é executada n vezes

```
1 int busca(int [] v, int n, int x) {
2     int i;
3     for (i = 0; i < n; i++){
4         if (v[i] == x) {
5             return i;
6         }
7     }
8     return -1;
9 }
```

Depende do valor de n

- n = 10 vs n = 10000

Busca sequencial e consumo de tempo

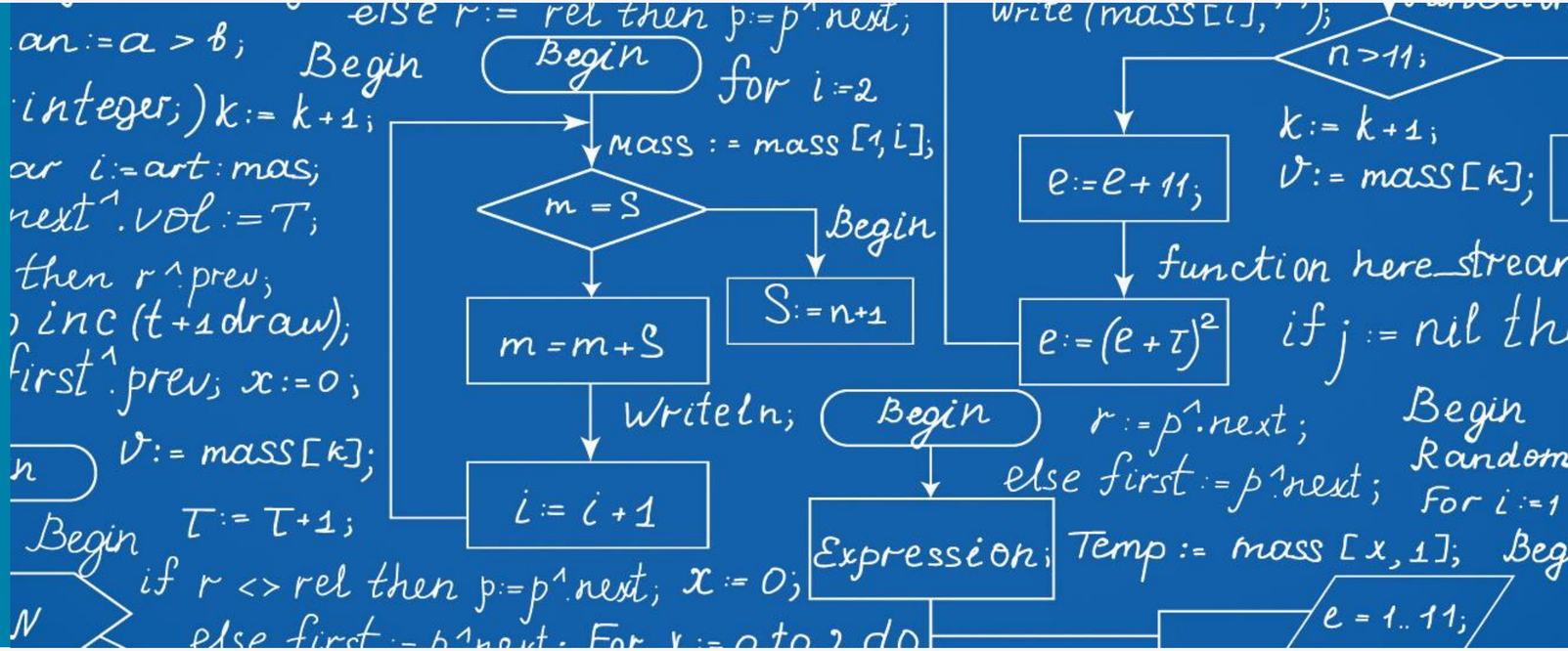
Queremos analisar algoritmos

Independentemente do computador onde ele for rodado

Em função do valor de n (a quantidade de dados)

Busca sequencial e consumo de tempo

Em geral, queremos analisar o pior caso do algoritmo, o que é sempre bom



- A análise do melhor caso pode ser interesse, mas é rara. Nem sempre seu algoritmo vai ser altamente rápido;
- A análise do caso médio é mais difícil
  - Normalmente é uma análise probabilística
  - Precisamos fazer suposições sobre os dados de entrada

Busca sequencial e consumo de tempo

Consumo de tempo por linha no pior caso:

- **Linha 2: tempo c_2 (declaração de variável)**
- **Linha 3: tempo c_3 (atribuições, acessos e comparação). No pior caso, essa linha é executada $n + 1$ vezes**

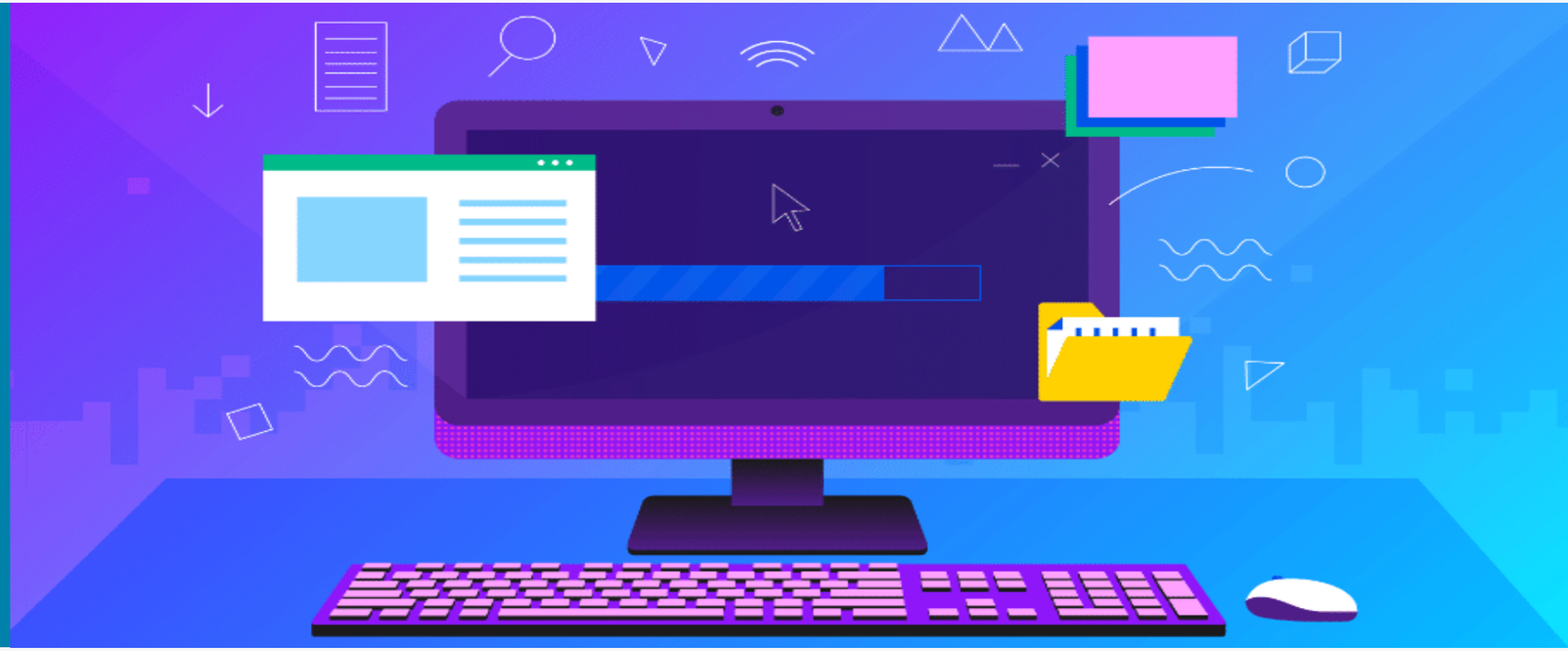
```
1 int busca(int [] v, int n, int x) {
2     int i;
3     for (i = 0; i < n; i++){
4         if (v[i] == x) {
5             return i;
6         }
7     }
8     return -1;
9 }
```

- **Linha 4: tempo c_4 (acessos, comparação e if). No pior caso, essa linha é executada n vezes**
- **Linha 5: tempo c_5 (acesso e return)**
- **Linha 8: tempo c_8 (return)**
- **O tempo de execução é menor ou igual a $c_2 + c_3.(n + 1) + c_4.n + c_5 + c_8$**

Busca sequencial e consumo de tempo

O tempo de execução é menor ou igual a:

$c_2 + c_3.(n+1) + c_4.n + c_5 + c_8$



- Cada c_i não depende de n, depende apenas do computador
- c_i Leva um tempo constante

Busca sequencial e consumo de tempo

Sejam:

- $a = c_2 + c_3 + c_5 + c_8$
- $\mathrm{b} = c_3 + c_4$
- $d = a + b$

Se $n \geq 1$, temos que o tempo de execução é menor ou igual a

$$= c_2 + c_3.(n+1) + c_4.n + c_5 + c_8$$
$$= c_2 + c_3 + c_5 + c_8 + (c_3 + c_4).n$$
$$= a + b.n \leq a.n + b.n = d.n$$

Isto é, o crescimento do tempo é linear em n

- **Se n dobra, o tempo de execução praticamente dobra**

Notação assintótica

Como vimos

existe uma constante d tal que, para $n \geq 1$

$$c_2 + c_3.(n+1) + c_4.n + c_5 + c_8 \leq dn$$

d não interessa tanto

- **d só depende do computador**
- **Estamos preocupados em estimar**

Notação assintótica

- O tempo do algoritmo é da ordem de n
- A ordem de crescimento do tempo é igual a de $f(n) = n$

```
1 int busca(int [] v, int n, int x) {
2     int i;
3     for (i = 0; i < n; i++){
4         if (v[i] == x) {
5             return i;
6         }
7     }
8     return -1;
9 }
```

Dizemos que:

$$c_2 + c_3.(n+1) + c_4.n + c_5 + c_8 = O(n)$$

Busca binária

Se realizarmos t chamadas, quanto vale t?

```
1  int busca_binaria(int [] dados, int l, int r, int x) {
2      int m = (l + r) / 2;
3      if (l > r){
4          return -1;
5      }
6      if (dados[m] == x){
7          return m;
8      } else {
9          if (dados[m] < x){
10             return busca_binaria(dados, l, m - 1, x);
11         } else{
12             return busca_binaria(dados, m + 1, r, x);
13         }
14     }
15 }
```

Busca binária

Se realizarmos t chamadas, quanto vale t?

- primeiro chamamos para n
- Depois para $\frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots, \frac{n}{2^{t-1}}$
- No pior caso, só paramos quando

  $\frac{n}{2^t} < 1 \leq \frac{n}{2^{t-1}}$
- Ou seja, $t \leq 1 + \log n$

Gastamos um tempo constante c em cada chamada

- operações aritméticas, comparações e return

Para $n \geq 1$, o consumo de tempo é no máximo

- $ct \leq c + c.\log n \leq 2.c \log n = O(\log n)$

Objetivos

Temos dois objetivos para analisar algoritmos

Entender o tempo de execução de um algoritmo

- **Exemplo: busca linear é** $O(n)$
- **Vamos dizer que o algoritmo é** $O(f(n))$

Comparar dois algoritmos

- **Busca linear é** $O(n)$ **e busca binária é** $O(\log n)$
- **Veremos que um algoritmo** $O(\log n)$ **é melhor que um** $O(n)$

Comparando funções

Queremos comparar duas funções f e g

- Queremos entender a velocidade de crescimento de f
- Queremos dizer que f cresce mais lentamente ou igual a g

f pode ser o tempo de execução do algoritmo e g uma função mais simples de entender

- $f(n) = c_2 + c_3.(n+1) + c_4.n + c_5 + c_6$ **e** $g(n) = n$
- $f(n) = 3n^2 + 10\log n$ **e** $g(n) = n^2$

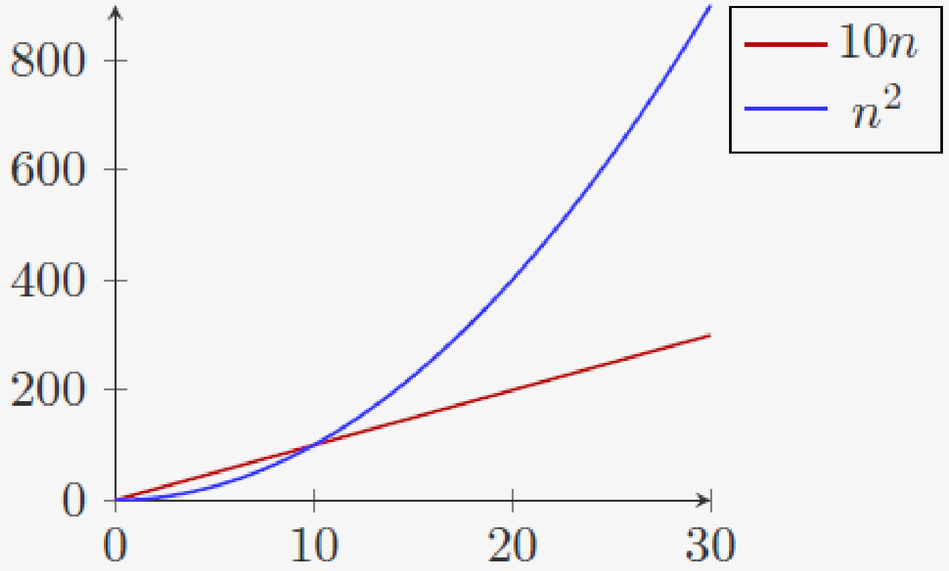
f e g podem ser os tempos de execução de dois algoritmos

$f(n) = dn$ **e** $g(n) = c + c\log n$

Primeira ideia

Comparar funções verificando se $f(n) \leq g(n)$ para todo n

Problema: $10n > n^2$ **para** $n < 10$

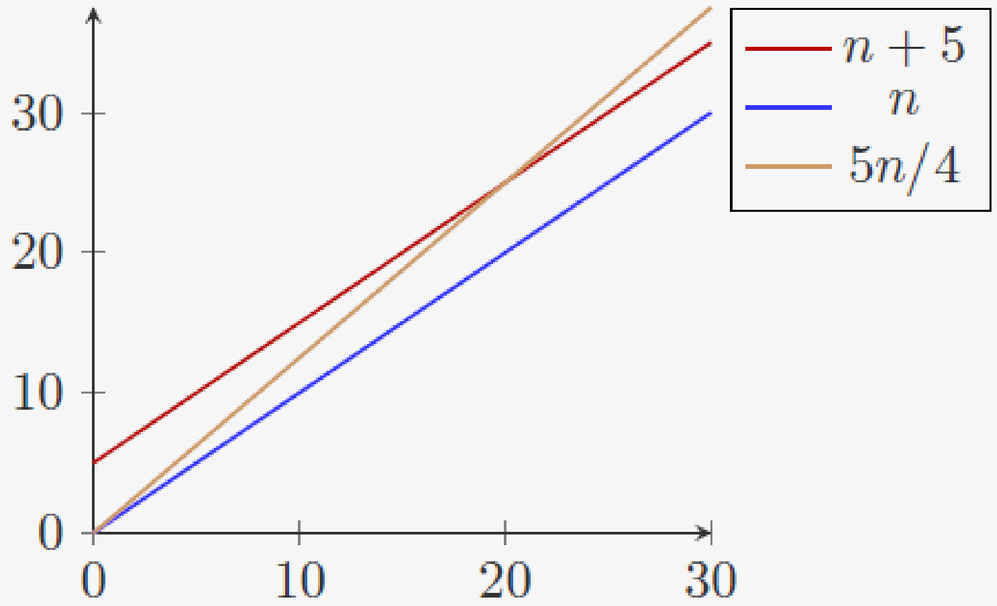


Solução: Ao invés de comparar todo n, comparar apenas n suficientemente grande

- **Para todo $n \geq n_0$ para algum n_0**

Segunda ideia

Comparar funções verificando se $f(n) \leq g(n)$ para $n \geq n_0$



Problema: $n + 5 > n$ **para todo** n

- **Mas a velocidade de crescimento das funções é o mesmo**
- **Constantes dependem da máquina onde executamos**
- **Vamos ignorar constantes e termos menos importantes**

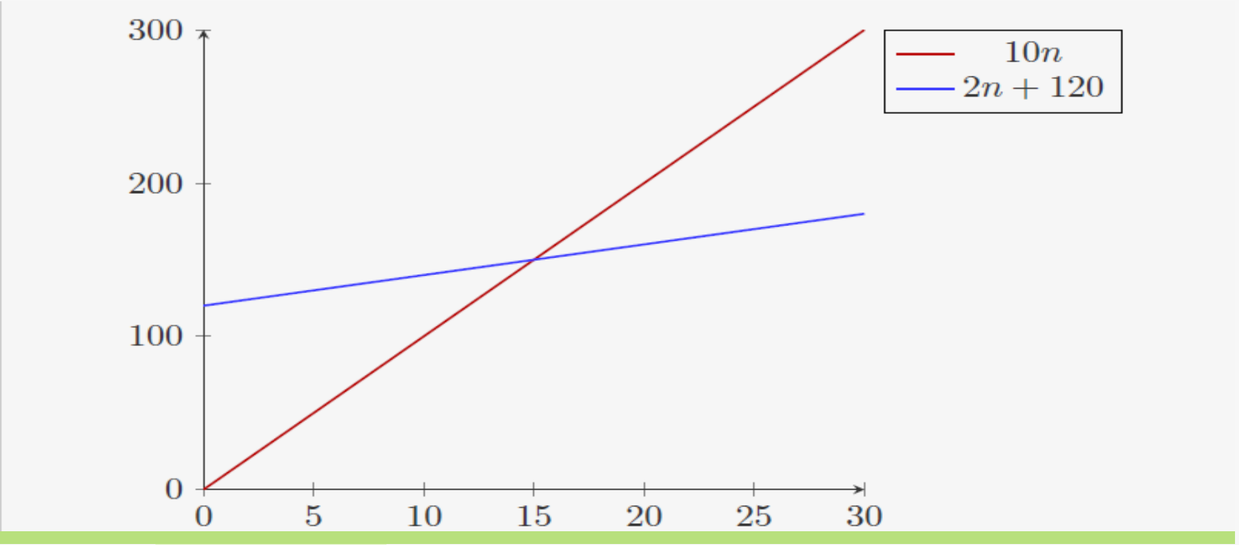
Notação assintótica

Dizemos que uma função $f(n) = O(g(n))$ se:

- **Existe uma constante c**
- **Existe uma constante n_0 tal que: $f(n) \leq c.g(n)$, para todo $n \geq n_0$**

$f(n) = O(g(n))$ **se, para todo n suficientemente grande, $f(n)$ é menor ou igual a um múltiplo de $g(n)$**

Exemplos

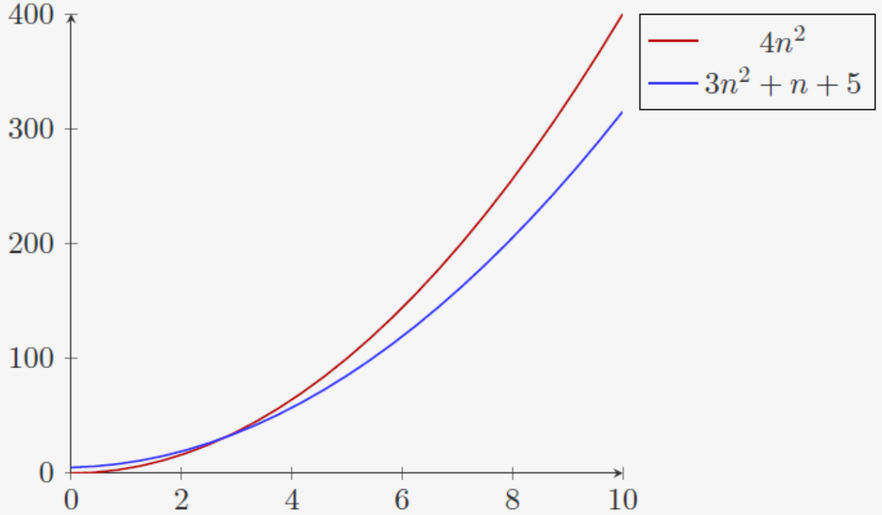


$$2n + 120 = O(n)$$

$$c = 10 \textbf{ e } n_0 = 15$$

$$3n^2 + n + 5 = O(n^2)$$

$$c = 4 \textbf{ e } n_0 = 4$$



Análise assintótica

Outros exemplos

- $1 = O(1)$
- $1.000.000 = O(1)$
- $5n + 2 = O(n)$
- $5n^2 + 5n + 2 = O(n^2)$
- $\log_2 n = O(\log_{10} n)$
- $\log_{10} n = O(\log_2 n)$

Nomenclatura e consumo de tempo

$O(1)$: tempo constante

- Não depende de n
- Ex: atribuição e leitura de uma variável
- Ex: operações aritméticas: +, -, *, /
- Ex: comparações (<, <=, ==, >=, >, !=)
- Ex: operadores booleanos (&&, &, ||, |, !)
- Ex: acesso a uma posição de um vetor

$O(\log n)$: logarítmico

- lg indica $\log_2$
- quando n dobra, o tempo aumenta em uma constante
- Ex: Busca binária
- Outros exemplos durante o curso

Nomenclatura e consumo de tempo

$O(n)$: linear

- quando n dobra, o tempo dobra
- Ex: Busca linear
- Ex: Encontrar o máximo/mínimo de um vetor
- Ex: Produto interno de dois vetores

$O(n^2)$: quadrático

- quando n dobra, o tempo quadriplica
- Ex: BubbleSort, SelectionSort e InsertionSort

$O(n \log n)$:

- quando n dobra, o tempo um pouco mais que dobra
- Ex: algoritmos de ordenação que veremos

$O(n^3)$: cúbico

- quando n dobra, o tempo octuplica

Um cuidado

O que significa dizer que o tempo de um algoritmo é $O(n^3)$?

- Para instâncias grandes $(n \geq n_0)$
- O tempo é menor ou igual a um múltiplo de n^3

Pode ser que o tempo do algoritmo seja $2n^2$...

- $2n^2 = O(n^3)$, mas ...
- $2n^2 = O(n^2)$

Ou seja, podemos ter feito uma análise "folgada"

- achamos que o algoritmo é muito pior do que é realmente

Exercício

Este exercício é apenas para praticar, não é para enviar. A prática leva ao aprendizado

1) Mostre que $n + \log n = O(n)$

2) Mostre que $15n = O(n \log n)$**, mas que** $n \log n \neq O(n)$

Essa análise é folgada, já que $15n = O(n)$

3) Mostre que $42n = O(n^2)$**, mas que** $n^2 \neq O(42n)$

Essa análise é folgada, já que $42n = O(n)$

Dúvidas, sugestões ou reclamações?

...

Alguma pergunta?

UEPB
Universidade
Estadual da Paraíba

Referências

1. JÚNIOR, Fábio Luiz Leite. **Estrutura de dados e algoritmos**. 2012. UEPB.

2. CORMEN, Thomas H. **Algoritmos: teoria e prática**. 3ª edição. Rio de Janeiro : Elsevier, 2012.

3. LEE, Orlando. Estrutura de dados. 2022. Unicamp.

4.

5.

UEPB
Universidade
Estadual da Paraíba